Metropolis Algorithm

Thursday, 13 January 2022 11:01 AM

Metropolis algorithm is an algorithm for generating a sequence of random numbers according to a given probability distribution $f(x)$.

The algorithm is applicable to multidimensional probability distributions i.e. $f(x_1, x_2, \ldots x_n)$

Only knowledge of the relative probabilities is needed.
i.e. Knowledge of the normalization factor is not required.

This is an important advantage, because in Statistical mechanics, we only know the relative probability

$$\text{i.e. } f(\vec{r}^N) \propto e^{-\beta U(\vec{r}^N)}$$

The normalization factor, which is the partition function, is not known.

All the above advantages makes the Metropolis Algorithm particularly suitable for molecular simulations

Markov Process

The Metropolis algorithm relies on a stochastic process called "Markov Process" to generate the sequence of random numbers

Property of a Markov Process
1) The probability of an event occuring in a given sequence only depends on the previous event.

Consider a stochastic process where the variable X_i has the following sequence of values

$$x_1, x_2, x_3 \ldots, x_n$$

In a Markov process, the value of the next random number, x_{n+1}, only depends on x_n.

In terms of transition probabilities

$$\text{Transition Probability} \equiv \Pr\left\{X^{(n+1)} = x_{n+1} \mid X^{(n)} = x_n\right\}$$

$$= \begin{cases} f(x_1, x_2, \ldots x_n) & \leftarrow \text{for any stochastic process} \\ f(x_n) & \leftarrow \text{for a Markov process} \end{cases}$$

In a Monte Carlo molecular simulation, we are interested in equilibrium states of the system.
At thermodynamic equilibrium, the probability density of the microstates does not change with time.

Hence, we would require the Markov Process to generate a stationary distribution of random numbers.
i.e. the probability density of X does not change as the process proceeds.

To get a stationary distribution, the Markov process has to satisfy the following condition

$$\Pr\left\{X = x \mid x' \neq x\right\} = \Pr\left\{X = x' \neq x \mid X = x\right\}$$

Probability of $X = x$ given that the previous number $x' \neq x$

Probability of $X = x' \neq x$ given that previous value is x

This condition is harder to implement.
Hence we impose a stronger condition that is easier to implement.

$$\Pr\left\{X = x \mid x'\right\} = \Pr\left\{X = x' \mid x\right\} \quad \leftarrow \text{Microscopic reversibility}$$

Probability of $X = x$ given that previous value $= x'$

Probability that $X = x'$ given that previous value is x.

The above condition is known as condition of detailed balance

Implementation

Problem: generate a sequence of random number for the variable X

according to the probability density $f(x)$.

Condition of detailed balance

$$Pr\{x^{(old)} \to x^{(new)}\} = Pr\{x^{(new)} \to x^{(old)}\}$$

$$Pr\{x^{(old)}\}\ Pr\{x^{(old)} \to x^{(new)} \mid x^{(old)}\} = Pr\{x^{(new)}\}\ Pr\{x^{(new)} \to x^{(old)} \mid x^{(new)}\}$$

$Pr\{x^{(old)}\} \rightarrow f(x^{(old)})$

$Pr\{x^{(old)} \to x^{(new)} \mid x^{(old)}\} \rightarrow \alpha(x^{(old)} \to x^{(new)})\ acc(x^{(old)} \to x^{(new)})$

$\alpha(x^{(old)} \to x^{(new)})$ ≡ Probability of proposing $x^{(new)}$ given current value is $x^{(old)}$

$acc(x^{(old)} \to x^{(new)})$ ≡ Probability of accepting the change from $x^{(old)}$ to $x^{(new)}$.

$$\therefore\quad f(x^{(old)})\,\alpha(x^{(old)} \to x^{(new)})\,acc(x^{(old)} \to x^{(new)}) = f(x^{(new)})\,\alpha(x^{(new)} \to x^{(old)})\,acc(x^{(new)} \to x^{(old)})$$

$$\therefore\quad \frac{acc(o \to n)}{acc(n \to o)} = \frac{f(n)\ \alpha(n \to o)}{f(o)\ \alpha(o \to n)} = r$$

In a Monte Carlo simulation, we design the functional form of "α". The acceptance probability, acc, should satisfy the above condition. There are many forms of acc, that satisfies the above condition. The most common form is

$$acc(o \to n) = \min\{1, r\}$$

if $r < 1$ then $acc(o \to n) = r$ & $acc(n \to o) = 1$ $\Rightarrow \frac{acc(o \to n)}{acc(n \to o)} = r$

if $r > 1$ then $acc(o \to n) = 1$ & $acc(n \to o) = 1/r$ $\Rightarrow \frac{acc(o \to n)}{acc(n \to o)} = \frac{1}{1/r} = r$

Example

Generate random number according to $f(x)$

Step 1: Start with an initial value : x_0

Step 2: Propose a new number x' according to the following probability density, α

$$\alpha[x^{(old)} \to x^{(new)}] = \begin{cases} \frac{1}{2\Delta x_m} & \text{if } x^{(old)} - \Delta x_m < x^{(new)} < x^{(old)} + \Delta x_m \\ 0 & \text{otherwise} \end{cases}$$

↑

Propose a new number $x^{(new)}$ uniformly in the interval $(x^{(old)} - \Delta x_m,\ x^{(new)} + \Delta x_m)$

The value of Δx_m is specified at the start of the simulation

Generate a uniform random number, u, between 0 and 1

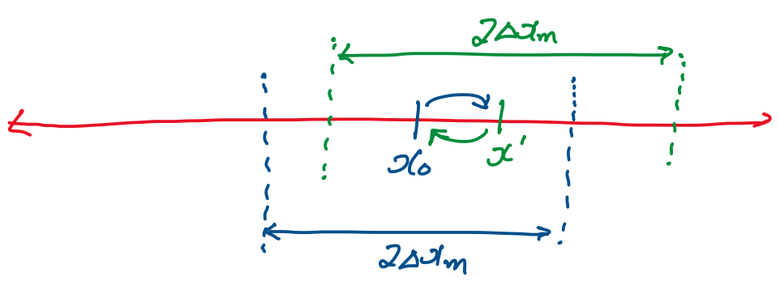
$$\Delta x = (2u - 1)\,\Delta x_m$$

$$x' = x_0 + \Delta x$$

Step 3: Compute $f(x_0)$ and $f(x')$

Step 4: Not that for the above 'α',

$$\alpha(x_0 \to x') = \alpha(x' \to x_0) = \frac{1}{2\Delta x_m}$$



$$\therefore\quad acc(x \to x') = \min\{1, f(x')\}$$

$\cdots \quad \alpha = \min\left(1, \frac{f(x')}{f(x_0)}\right)$

Step 5: Accept the new number according to the above probability

if $f(x') > f(x_0)$, then $x_1 = x'$ ⇒ always accept the change

if $f(x') < f(x_0)$, then define $r = \frac{f(x')}{f(x_0)}$

generate a number 'u' between 0 and 1

if $u < r$, accept the change i.e. $x_1 = x'$

else reject the change i.e. $x_1 = x_0$

Step 6: Repeat steps 2 – 5

one example of sequence of numbers

$x_0, x_1, x_1, x_2, x_3, x_3, x_3, x_4, x_5 \cdots$

change accepted

change rejected